PageRank

355199 Ludovica Ivaldi - 354965 Arianna Giulia Biancone

1 Introduction

PageRank is an algorithm developed by Google to evaluate the importance of webpages by analysing their link patterns. Since search engines must decide which pages to display first, PageRank provides a principled way to order webpages based on this structure. The key idea is that a page gains importance from the importance of the pages linking to it, so influence propagates through the network.
Mathematically, because each page's score depends on the scores of the pages pointing to it, the definition becomes inherently recursive. When this dependency is written simultaneously for all pages, it leads to the eigenvector equation

$$Ax = x,$$

whose dominant eigenvector represents the PageRank vector.

In this report we investigate how this mechanism behaves in different scenarios. We begin with small examples to understand how the link matrix A is constructed and how changes to the graph, such as adding a new page or modifying the link structure, affect the resulting ranking.
We then introduce the modified matrix

$$M = (1 - m)A + mS,$$

which theoretically ensures a unique and stable ranking even when the network is not fully connected (i.e. there are sub-components in the web graph).
Furthermore, since computing PageRank requires finding the dominant eigenvector of the link matrix, we analyse the convergence of the power method, which is the standard technique used to approximate the PageRank vector.
Finally, we implement the PageRank algorithm and apply it to a large real dataset, working with sparse matrices and handling the presence of dangling nodes.

2 Exercises

2.1 Exercise 1

Suppose the people who own page 3 in the web of Figure 1 are infuriated by the fact that its importance score, computed using formula (2.1), is lower than the score of page 1. In an attempt to boost page 3's score, they create a page 5 that links to page 3; page 3 also links to page 5. Does this boost page 3's score above that of page 1?

The core idea of this exercise is to demonstrate how it is possible to increase a page's PageRank score introducing a strategic link to a dummy page.
The fundamental mechanism of how PageRank distributes link "votes" is governed by the importance of the linking pages. It is not merely the number of inbound links that matters, but the quality of those links. The mathematical basis for this weighting scheme is given by the PageRank equation (Formula 2.1), where the score of each linking page j to page k is divided among all the pages it points to:

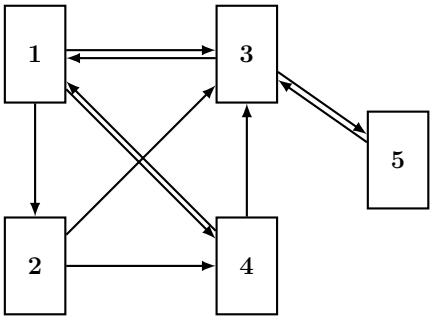
$$x_k = \sum_{j \in L_k} \frac{x_j}{n_j}$$

where:

- x_k is the score of Page k.
- L_k is the set of pages that link to Page k.
- x_j is the score of the linking Page j.
- n_j is the total number of outgoing links from Page j.

The contribution received by a page significantly increase when an important page links to only a few other pages. Conversely, if a page distributes its score among many outgoing links, the weight received by each target page is diluted.

By creating page 5 and letting it link only to page 3, page 3 receives the full score of page 5. Since page 3 also links back to page 5, this mutual reinforcement increases the score of page 3.



$$A' = \begin{pmatrix} 0.0 & 0.0 & 1/2 & 1/2 & 0.0 \\ 1/3 & 0.0 & 0.0 & 0.0 & 0.0 \\ 1/3 & 1/2 & 0.0 & 1/2 & 1.0 \\ 1/3 & 1/2 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 1/2 & 0.0 & 0.0 \end{pmatrix}$$

Figure 1: Graph and link matrix for Exercise 1

Rank Results – Fig. 2.1 without Page 5

Rank	Page	Score
1	1	0.387097
2	3	0.290323
3	4	0.193548
4	2	0.129032

(a) Fig. 2.1 without Page 5

Rank Results – Fig. 2.1 with Page 5

Rank	Page	Score
1	3	0.367347
2	1	0.244898
3	5	0.183673
4	4	0.122449
5	2	0.081633

(b) Fig. 2.1 with Page 5

Figure 2: PageRank results for the original graph and for the graph with the added page.

By comparing the two rankings, we observe that the introduction of page 5 changes the distribution of scores. As expected, Page 3 benefits from the additional incoming link and becomes the top-ranked page, while the relative order of the other pages changes only moderately.

2.2 Exercise 11

Consider again the web in Figure 2.1, with the addition of a page 5 that links to page 3, where page 3 also links to page 5. Calculate the new ranking by finding the eigenvector of M(corresponding to $\lambda = 1$) that has positive components summing to one. Use $m = 0.15$.

This exercise shows that computing the PageRank scores using the modified stochastic matrix $M = (1-m)A+mS$, instead of the original link matrix A, does not alter the resulting ranking of the pages. Although the eigenvector associated with the dominant eigenvalue $\lambda = 1$ changes since matrix A is structurally different from matrix M, the ordering of the pages (the ranking) is preserved.

In general,the introduction of matrix M is necessary to address the presence of multiple connected components. When the web is not fully connected, the ranking obtained from matrix A is not unique, since it is not possible to compare the scores of two pages located in different subwebs.
Mathematically, this problem corresponds to the number of linearly independent vectors associated

with the largest eigenvalues: in the case of matrix A, this space has a dimension greater than 1 and therefore admits more than one linearly independent eigenvector (rank); while in the case of M, the paper demonstrates that its dimension is equal to 1, so the rank is unique.

However, the web in Figure 2.1 with the addition page 5, the web remains connected. Therefore, our purpose is to show that replacing matrix A with matrix M does not lead to an incorrect ranking.

Rank Results – Fig. 2.1 with Page 5

Rank	Page	Score
1	3	0.367347
2	1	0.244898
3	5	0.183673
4	4	0.122449
5	2	0.081633

(a) Fig. 2.1 with Page 5, without using M

Rank Results – Fig 2.1 with Page 5, using M

Rank	Page	Score
1	3	0.348895
2	1	0.237144
3	5	0.178285
4	4	0.138490
5	2	0.097185

(b) Fig. 2.1 with Page 5, using M

Figure 3: PageRank results for the original graph and for the graph with the added page.

2.3 Exercise 14

For the web in Exercise 11, compute the values of $\|M^k x_0 - q\|_1$ *and* $\frac{\|M^k x_0 - q\|_1}{\|M^{k-1} x_0 - q\|_1}$ *for* $k = 1, 5, 10, 50$*, using an initial guess* x_0 *not too close to the actual eigenvector* q *(so that you can watch the convergence). Determine* $c = \max_{1 \le j \le n} |1 - 2 \min_{1 \le i \le n} M_{ij}|$ *and the absolute value of the second largest eigenvalue of* M*.*

In this exercise, we implemented the power method starting from a normalised initial vector x_0.
At each step, we computed the ℓ_1-norm of the error between the current iterate and the normalised dominant eigenvector q, which was accurately computed using the `eig` function from the `SciPy` library.

In particular, we computed the approximation error after $k = 1, 5, 10, 15, 20, 25, 50, 60, 71$ iterations, and evaluated the convergence rate as the ratio between the error at iteration k and the error at iteration $k - 1$. The results show that:

- Between iterations $k = 15$ e $k = 70$, the convergence rate stabilises around the value of the second-largest eigenvalue, 0.611, as theoretically expected for the power method.
- At iteration $k = 71$, the algorithm can be considered converged: the error at the last iteration coincides with that of the previous one, indicating that the approximation of the dominant eigenvector no longer changes.

As required by the exercises, we also computed the constant c, which can be interpreted as an upper bound on the convergence speed. In our case, we obtained

$$c = \max_{1 \le j \le n} \left| 1 - 2 \min_{1 \le i \le n} M_{ij} \right| = 0.94 > 0.611$$

which is consistent with the results reported in the paper.
This bound can be understood by considering the worst possible column of the matrix M. By construction, each column of M sums to 1, and therefore the smallest entry that can appear in a column is m/n. Under this worst-case assumption, the constant c can be written as

$$c = 1 - 2\frac{m}{n} = 0.94.$$

This expression highlights the conservative nature of the bound. In fact, c is derived from worst-case assumptions on the entries of M and therefore provides a pessimistic estimate of the true convergence rate. A more accurate indicator is given by the absolute value of the second-largest eigenvalue $|\lambda_2|$, which matches the empirical convergence observed in the iteration.

3 PageRank

In this part we implement the PageRank algorithm in order to compute the dominant eigenvector of the link matrix A, which determines the final ranking of the pages.
We first apply the algorithm on a real large database (Hollins.dat) and then on the two Figures of the paper.
The most suitable technique to approximate this eigenvector is the power method: starting from an initial score vector, the method applies the link matrix repeatedly and normalises the output at each step. Once the updates no longer changes the vector appreciably, the iteration has converged, and the resulting vector is an approximation of the dominant eigenvector.
This approach fits our setting well because it only requires matrix–vector products, and can therefore be implemented efficiently when the link matrix is sparse.

3.1 Hollins

The PageRank algorithm is first tested on the Hollins dataset, which consists of 6012 nodes, each representing a web page, and a set of directed edges corresponding to hyperlinks between pages. From this structure, the link matrix A is constructed.
A notable characteristic of the Hollins dataset is the presence of a large number of **dangling nodes**, i.e. pages with no outgoing links. In particular, 3189 nodes fall into this category, accounting for more than half of the total number of pages. This structural property has important implications for the PageRank computation, as it affects both the stochasticity of the link matrix and the convergence behavior of the power method.
For this reason, the standard PageRank formulation introduced in the original Google paper must be adapted in order to correctly handle dangling nodes and ensure a well-defined ranking.
In the following sections, we describe the sparse storage format adopted for the link matrix, the PageRank algorithm and its modifications, and finally the results obtained on the Hollins dataset.

3.1.1 Sparse Matrix Storage Format

The link matrix A of the Hollins dataset is stored in Compressed Sparse Row (CSR) format. This choice is motivated by the fact that, within the power method, the only operation involving matrix A is matrix–vector multiplication. For this purpose, CSR represents the most efficient and appropriate storage format.
Matrix A has a sparsity of 99.934% (equivalently, a density of 0.066%), meaning that only a small fraction of its entries are non-zeros. Exploiting this sparsity is therefore crucial to reduce both memory usage and computational cost.
In the CSR format, the array `AA` stores the values of the N_z non-zero entries of the matrix, while the array `JA` contains their corresponding column indices. The array `IA` identifies (with a comulative sum) the starting position in `AA` and `JA` of each matrix row. During matrix–vector multiplication, each row of A is processed by accessing only its non-zero entries; the column indices stored in `JA` are used to index the corresponding elements of the vector x, which are then multiplied by the values stored in `AA`.
As a result, the CSR format minimizes unnecessary operations on zero entries and ensures an efficient implementation of the power method.

3.1.2 Implemented Algorithm for PageRank

As previously mentioned, the ranking of the pages is obtained by computing the dominant eigenvector of the link matrix A. The most suitable technique to approximate this eigenvector is the power method. This method is particularly convenient in our setting, since it relies only on sparse matrix–vector multiplications. As observed in the paper, the power method converges provided that the matrix used in the iteration is column–stochastic.
When the link matrix satisfies the column–stochastic property, the power method can in fact be applied directly to compute the ranking. In this ideal situation, where every page has at least one outgoing link, the update performed at each step takes the form

$$x^{(k+1)} = (1-m)A\,x^{(k)} + mS\,x^{(k)},$$

which is simply the power method applied to the matrix $M = (1-m)A + mS$. The term $(1-m)Ax^{(k)}$ propagates the scores along the outgoing links, while the term $mSx^{(k)}$ redistributes a fraction of the score uniformly across all pages. Thanks to the teleportation component mS, every page remains reachable, so disconnected groups of pages (subwebs) do not pose any difficulty for convergence.
However, in real web graphs such as the Hollins dataset, the link matrix A is rarely column–stochastic. The main obstacle is the presence of *dangling nodes*, i.e. pages with no outgoing links, which appear as zero columns in A. Such columns cause the iteration to accumulate score on these pages rather than converge to a meaningful vector, and therefore require an explicit correction in the update rule.

A natural idea would be to correct this problem by modifying the matrix A itself. For dangling nodes, one could replace each zero column by a uniform column whose entries are equal to $1/n$. This corresponds to assuming that, when a page has no outgoing links, a surfer can move to any page in the network with equal probability. However, this modification would make the matrix A dense, destroying sparsity and greatly increasing the computational cost, especially for large datasets such as the Hollins graph.
To avoid turning A into a dense matrix, we leave its entries unchanged and incorporate the necessary corrections directly into the update of the power method. This produces an iteration that is equivalent to applying the corrected matrix, but without ever constructing it explicitly.

The main corrections we made to the update of the power method are the following:

1) Dangling nodes. To correct the behaviour of dangling nodes we would ideally replace each zero column of A with a uniform column whose entries are all equal to $1/n$. To avoid making A dense, a correction matrix D could be constructed containing $1/n$ in the columns associated with dangling nodes and all zeros in the remaining columns.
However, since all rows of D would be identical by construction, multiplying D by the vector $x^{(k)}$ would produce a vector in which every component has the same value. Because the result of the multiplication depends only on this repeated row, constructing the full matrix D offers no advantage and we only store an indicator vector d, where $d_j = 1/n$ if page j is a dangling node and 0 otherwise.

The uniform redistribution of the probability associated with dangling nodes can then be efficiently incorporated through the term

$$(1-m)\,(d^\top x^{(k)})\,\mathbf{1},$$

which is algebraically equivalent to multiplication by the matrix D and allows the sparsity of the matrix A to be preserved.

2) Subwebs. A similar consideration applies to the teleportation term. The matrix S introduced in the paper is fully dense, with all entries equal to $1/n$, and all its rows are identical. Hence, it is sufficient to use a single row vector s with entries equal to $1/n$.
Multiplying this row vector by the PageRank iterate $x^{(k)}$ yields

$$s\,x^{(k)} = \frac{1}{n}\sum_{i=1}^{n} x_i^{(k)}.$$

Since the power method preserves the normalization $\|x^{(k)}\|_1 = 1$, the product reduces to

$$s\,x^{(k)} = \frac{1}{n}.$$

As a result, the multiplication $Sx^{(k)}$ produces a constant vector, and the teleportation contribution can be incorporated by directly adding

$$\frac{m}{n}\mathbf{1},$$

which has exactly the same effect while preserving sparsity and computational efficiency.

Final update. Combining the propagation through the actual links with the dangling–node correction and the teleportation contribution gives the iteration used in our implementation:

$$x^{(\mathrm{k}+1)} = (1-m)Ax^{(\mathrm{k})} + (1-m)(d^\top x^{(\mathrm{k})})\,\mathbf{1} + \frac{m}{n}\,\mathbf{1}.$$

This formulation preserves the sparsity of A, avoids the creation of large dense matrices, and ensures that the iteration converges even in the presence of dangling nodes and subwebs.

Computational cost. Let n denote the number of nodes in the graph and let $\mathrm{Nz}(A)$ be the number of non-zero entries of the link matrix A. In our case, $A \in R^{6012\times 6012}$ is stored in CSR format and contains $\mathrm{Nz}(A) = 23875$ non-zero elements, corresponding to an extremely sparse structure.
At each iteration of the power method, the dominant computational cost arises from the link propagation term, which requires the sparse matrix–vector multiplication.
Since A is stored in CSR format, only the non-zero entries are accessed during the computation. Therefore, the cost of a single matrix–vector product is proportional to the maximum number of non-zero elements in each row of A, and can be bounded by

$$O(\mathrm{N_r} * n),$$

which is significantly lower than the $O(n^2)$ cost associated with dense matrices.
The remaining terms of the PageRank update, which account for dangling nodes and teleportation, have linear complexity in n and are therefore negligible with respect to the sparse matrix–vector multiplication.

As a result, the per-iteration computational complexity of the algorithm is

$$O(\mathrm{N_r} * n).$$

Since the power method requires k iterations to converge, the overall computational complexity of the PageRank computation is

$$O\big(k * \mathrm{N_r} * n\big).$$

3.1.3 Results

We applied the implemented PageRank algorithm to the Hollins dataset using the corrected power method described in the previous section. The resulting ranking is stable and consistent, indicating that the adopted modifications successfully handle the structural properties of the graph.

Figure 4 reports the top–20 pages obtained from the algorithm.

PageRank – Hollins dataset (Top 20 Pages)

Rank	Page	Score	URL
1	2	0.019879	http://www.hollins.edu/
2	37	0.009288	http://www.hollins.edu/admissions/visit/visit.htm
3	38	0.008610	http://www.hollins.edu/about/about_tour.htm
4	61	0.008065	http://www.hollins.edu/htdig/index.html
5	52	0.008027	http://www.hollins.edu/admissions/info-request/info-request.cfm
6	43	0.007165	http://www.hollins.edu/admissions/apply/apply.htm
7	425	0.006583	http://www.hollins.edu/academics/library/resources/web_linx.htm
8	27	0.005989	http://www.hollins.edu/admissions/admissions.htm
9	28	0.005572	http://www.hollins.edu/academics/academics.htm
10	4023	0.004452	http://www1.hollins.edu/faculty/saloweyca/clas%20395/Sculpture/sld001.htm
11	29	0.004385	http://www.hollins.edu/grad/coedgrad.htm
12	5254	0.003778	http://www1.hollins.edu/faculty/saloweyca/clas%20395/Sculpture/tsld001.htm
13	3227	0.003742	http://www1.hollins.edu/faculty/saloweyca/clas%20395/Sculpture/index.htm
14	40	0.003526	http://www.hollins.edu/alumnae/alumnae.htm
15	3834	0.003398	http://www1.hollins.edu/faculty/saloweyca/clas%20395/BronzeAGe/sld001.htm
16	822	0.003337	http://www1.hollins.edu/classes/dance/website/home.htm
17	4075	0.003334	http://www1.hollins.edu/faculty/saloweyca/clas%20395/Sculpture/sld053.htm
18	73	0.003318	http://www.hollins.edu/calendar
19	132	0.003229	http://www.hollins.edu/calendar/index.html
20	81	0.003092	http://www.hollins.edu/admissions/financial/finaid.htm

Figure 4: Hollins dataset ranking

The ranking obtained from the Hollins dataset shows a pattern that is typical of real-world university websites. The top pages all correspond to highly connected, central sections of the website such as the homepage, admissions pages, and academic program pages. These pages serve as major entry points and tend to receive a large number of incoming links from the rest of the website, which naturally increases their PageRank score.

Overall, the results are consistent with the structure of a university website, where navigation hubs and frequently referenced informational pages dominate the ranking.

3.2 Figure 2.1

We then applied the algorithm to the small web graph shown in Figure 2.1. In this case the structure of the graph is explicit, so we can directly compare the ranking with the expected influence of each page.

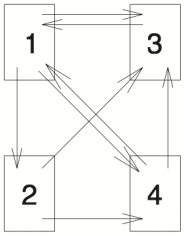


Figure 5: Graph of Figure 2.1

The computed ranking using our algorithm is shown in Figure 6.

```
PageRank - Fig 2.1
------------------------------------
Rank   Page   Score
------------------------------------
1      1      0.368151
2      3      0.287962
3      4      0.202078
4      2      0.141809
------------------------------------
```

Figure 6: Figure 2.1 ranking using our algorithm

In particular, Page 1 receives the largest score, consistently with the fact that several "important" pages point to it.
As the ranking shows, the result obtained with our corrected power method matches the ordering presented in Figure 2(a), which was computed by directly evaluating the dominant eigenvector of the link matrix.

3.3 Figure 2.2

Finally, we applied our PageRank implementation to the graph shown in Figure 2.2 of the paper, that contained a subweb.

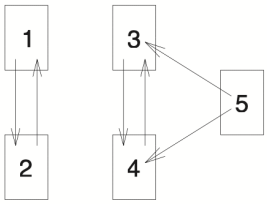


Figure 7: Graph of Figure 2.2

The ranking obtained with our implementation is shown in Figure 8.

PageRank on Fig 2.2

Rank	Page	Score
1	3	0.285000
2	4	0.285000
3	1	0.200000
4	2	0.200000
5	5	0.030000

Figure 8: Figure 2.2 ranking using our algorithm

As we can observe from the graph, pages 3 and 4 form a subweb, they point to one another and also receive additional incoming links from page 5. This closed linking structure allows them to reinforce each other, and the ranking produced by our method assigns them the highest scores, as expected.
Pages 1 and 2 also link to one another, but unlike the subweb they do not receive any links from outside their two–node component. Their scores are therefore lower than those of pages 3 and 4, but remain higher than the score of page 5.
Finally, page 5 has only outgoing links and no incoming ones, so it receives the smallest ranking value. The overall ordering is fully consistent with the link structure, confirming that the corrected power method captures the relative importance of the pages correctly.